ShadowMem

Volatile Memory Forensics and Fileless Malware Detection - Complete Project Status Dossier and Session-Planning Context Pack

Generated 2026-10-30 23:05. Purpose: give an AI assistant (Claude) complete context in one document so it can plan an entire work session - architecture, current state, gaps, and a prioritized backlog are all included.

Field	Value
Project	ShadowMem - Tier-1 SOC-style memory forensics dashboard and detection engine
Domain	Windows user-mode volatile memory forensics; fileless/in-memory malware detection (MITRE ATT&CK; T1055, T1620)
Language / Runtime	Python 3.14 (.venv on Windows), stdlib ctypes for Win32 APIs
Core stack	FastAPI 0.141.1 + Uvicorn, scikit-learn 1.9.0, numpy 2.5.2, pandas 3.0.5, psutil 7.2.2
Frontend	Single-page SOC dashboard; embedded HTML in app.py plus a standalone index.html (Tailwind CDN, dark HUD theme)
ML model	shadowmem_model.pkl - sklearn Pipeline (RobustScaler + HistGradientBoostingClassifier), 7-feature input
OS target	Windows only (VirtualQueryEx / OpenProcess / VirtualAlloc); admin rights required for system processes
Repo state	Local folder, no git history - status below inferred entirely from the code present on disk
Scale	9 Python/HTML source files, ~3,449 lines of code

1. Executive Summary

ShadowMem is a defensive security tool that detects fileless malware living only in RAM. Instead of scanning disk files (defeated by in-memory injection) or dumping whole physical memory (slow, gigabytes, needs a machine freeze), it walks each running process's virtual address space out-of-band via the Win32 VirtualQueryEx API, counts suspicious memory regions - RWX (read-write-execute) pages and unbacked executable MEM_PRIVATE regions - and feeds a 7-feature vector into a trained scikit-learn classifier. Results stream into a cyber-defense-operations-center (SOC) style web dashboard with threat scores, filterable process tables, a forensic dossier modal per process, and a live telemetry ticker.

What makes it notable:

- Real scanning pipeline: it genuinely opens processes and enumerates VADs (VirtualQueryEx), not just synthetic data - the dashboard reflects the live system.
- ML with forensic reasoning: training data is generated from documented per-technique memory signatures (reflective DLL injection, process hollowing, shellcode beacons, thread hijacking) with deliberate class overlap and noise so metrics stay honest.
- JIT false-positive control: browsers/Electron runtimes legitimately allocate RWX and unbacked executable memory, so known JIT engines are allowlisted and score-capped at 25.
- Safe self-validation: test_injector.py allocates a real 128 KB RWX region with a harmless NOP-sled payload in its own process so end-to-end detection can be demoed without ever injecting into another process or executing untrusted code.

The repo previously ran two parallel stacks: a REAL stack (memoryscanner.py -> app.py + embedded dashboard, scored by the trained model) and a DEMO stack (server.py + index.html, streaming synthetic WebSocket telemetry/alerts for UI showcase). The WebSocket demo stack has now been successfully unified into app.py, and server.py is retired to tests/mocks/ as a test-only mock fixture (complete - see Section 5).

2. File Inventory and Roles

File	Size	Role	Status
app.py	143 LOC	Main FastAPI app: /api/scan (real scan + ML scoring + JIT allowlist), / serves the embedded SOC dashboard (HUD, radar canvas, dossier modal, telemetry ticker, JSON export)	Done - core of the real stack

File	Size	Role	Status
memoryscanner.py	211 LOC	User-mode scanner: OpenProcess + VirtualQueryEx walk of every committed MEM_PRIVATE region; counts RWX and unbacked executable regions; psutil for names/threads	Done - features 1-4 real, 5 and 7 not implemented
shadowmem_model.pkl	38023 LOC	Trained sklearn Pipeline artifact (RobustScaler + HistGradientBoostingClassifier), predict_proba over the 7-feature vector	Done - trained on synthetic data only
train_model.py	616 LOC	Training pipeline: 7 synthetic forensic classes with documented distributions, noise and overlap injection, RobustScaler+HGB (or RandomForest), 5-fold stratified CV, permutation importance, final fit, pickles the model	Done - self-contained and reproducible (seed 42)
detecter.py	115 LOC	CLI twin of the web scoring logic: prints a PID table with verdicts (MALICIOUS >= 70, SUSPICIOUS >= 35, JIT capped at 25)	Done - shares the hardcoded-0.0 limitation
test_injector.py	328 LOC	Safe validation harness: VirtualAlloc 128 KB PAGE_EXECUTE_READWRITE in its own process, stages NOP sled + RET (never executes it), spawns a worker thread, cleans up with VirtualFree on Ctrl+C	Done - Windows-only, needs admin for best scan visibility
tests/mocks/server.py	304 LOC	SEPARATE demo FastAPI app: WebSocket /ws/alerts streams synthetic telemetry/alerts/memory maps; POST /simulate/alert for manual triggers; CORS wide open; 'XGBoost' is a stubbed confidence generator	Done as demo - synthetic only, disconnected from real stack
index.html	1456 LOC	Standalone 1,449-line demo dashboard consuming server.py's WebSocket (attack map, alert feed, memory-map visualizer)	Done as demo - pairs with server.py, not app.py
trigger_test_event.py	30 LOC	Posts a cobalt_strike_reflective_injection scenario to server.py's /simulate/alert	Done
launch_shadowmem.bat	39 LOC	Activates .venv, opens browser, starts uvicorn app:app on 127.0.0.1:8000	Done - real stack launcher
make_guide.py	246 LOC	Generates presentation_guide.html - a viva/presentation master guide (speaker scripts, OS internals, Q&A;, demo sequence)	Done - note: guide text mentions SMOTE and XGBoost; the code actually uses class_weight balancing and HistGradientBoosting/RandomForest

Not present but expected: requirements.txt, README.md, any automated tests, and any benchmark/profiling script. No git repository is initialized in this folder.

3. Architecture and Data Flow

3.1 Real stack (app.py + memoryscanner.py + model)

GET /api/scan -> memoryscanner.scan_all_processes() -> for each accessible PID: psutil name/thread count, then OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ) and a VirtualQueryEx loop over the address space. Every MEM_COMMIT + MEM_PRIVATE region with protection PAGE_EXECUTE_READWRITE increments rwx_count and adds RegionSize to total_rwx_bytes; PAGE_EXECUTE_READ or PAGE_EXECUTE_READWRITE increments unbacked_exec_count. app.evaluate_process() then builds the 7-feature vector [rwx_count, unbacked_exec_count, total_rwx_bytes, num_threads, thread_unbacked_start, is_known_jit_parent, vad_protection_anomaly] - with the last-but-two and last features hardcoded to 0.0 in this scanner - and calls model.predict_proba; the malicious column becomes the threat score (0-100). Known JIT engines are score-capped at 25 and labeled 'Legitimate JIT Engine'; otherwise >= 70 is 'Malicious (Injected)', >= 35 'Suspicious', else 'Clean'. If the model file is missing, a deterministic fallback heuristic is used (8*RWX + 15*unbacked + MB + 0.2*threads, capped at 100). Results are sorted by score and returned with a summary block; the embedded dashboard polls this endpoint on demand.

3.2 WebSocket streaming (hosted in app.py; server.py = mock fixture)

WebSocket streaming -- live alerts and memory_map broadcasts -- is now hosted entirely in app.py: the periodic scan ticker and POST /simulate/alert push alert frames and fresh memory_map frames over /ws/alerts from real scan data. server.py is no longer a live server; it is preserved only as a mock fixture under tests/mocks/ for testing purposes, serving synthetic telemetry, memory_map, and alert frames with MITRE-tagged scenarios on port 8082 so it can never collide with app.py on 8000.

3.3 Completed unified architecture

One FastAPI app now exposes both the real scanner results and the WebSocket feed: app.py handles real scan diffing (threat transitions between scans) and broadcasts real memory_map frames from live region data over /ws/alerts, alongside its rich dashboard UI. The former server.py streaming role lives in app.py; server.py is retired to tests/mocks/ as a test-only mock (complete - see Section 5).

4. Component Deep-Dives

4.1 Scanner internals (memoryscanner.py, 74 lines)

- Handles: OpenProcess with PROCESS_QUERY_INFORMATION (0x0400) | PROCESS_VM_READ (0x0010); non-admin processes return 'Access Denied' and are skipped silently.
- Walk: VirtualQueryEx in a while-loop advancing by mbi.RegionSize until it returns 0; MEMORY_BASIC_INFORMATION struct mirrors the Win32 layout.
- Signal: MEM_COMMIT + MEM_PRIVATE + (PAGE_EXECUTE_READWRITE 0x40 or PAGE_EXECUTE_READ 0x20) = unbacked executable region; RWX subset feeds rwx_count and total_rwx_bytes.
- Gaps: no read of region contents (no MZ/PE carving), no thread start-address resolution, no VAD protection-transition history, no MEM_IMAGE vs MEM_MAPPED distinction beyond Type.

4.2 Scoring engine (app.py evaluate_process / detecter.py)

- JIT allowlist (12 entries): chrome, msedge, firefox, code, discord, antigravity, slack, teams, electron, brave, opera, vivaldi (.exe, lowercase match). Cap: min(score, 25.0).
- Thresholds: MALICIOUS >= 70, SUSPICIOUS >= 35, else CLEAN. Dashboard posture flips to 'Threat Detected' when any malicious verdict exists.
- Fallback heuristic when shadowmem_model.pkl is absent - keeps the demo working without a model but is not calibrated to the trained model's scale.
- Limitation: allowlist is by image name only - renaming malware to chrome.exe evades the cap; path/signature verification is a TODO.

4.3 ML pipeline (train_model.py)

Sub-class (share)	Key distributions	Forensic rationale
benign_daemon (34%)	rwx ~ Poisson(0.15); threads ~ N(8,4); no unbacked starts	Near-zero RWX; exec regions backed by disk images
benign_gui (24%)	threads ~ N(28,10) - high on purpose; rwx ~ Poisson(0.3)	Teaches model 'high thread count alone != malicious'
benign_jit (20%)	rwx ~ N(18,8); unbacked ~ N(9,5); ~4 MB RWX cache; vad flips ~ N(3,2)	The classic false-positive source; RW->RX sealing, no remote threads
mal_reflective (6%)	unbacked ~ N(4,2); ~180 KB private exec; 1-2 unbacked thread starts	Manually mapped DLL: MEM_PRIVATE sections, late RW->RWX flip
mal_hollow (5%)	unbacked ~ N(7,3); ~900 KB unpack; vad flips ~ N(5,2.5); low threads	Image unmapped and rewritten; multi-stage re-protection burst
mal_shellcode (6%)	small: rwx ~ N(1.5,1); ~45 KB; ~1 thread start in shellcode	Beacon staged directly RWX; few or no protection flips
mal_hijack (5%)	rwx tiny; thread starts ~ Poisson(0.2) - reuses existing thread	SetThreadContext/APC injection evades new-thread heuristics

- Realism: Gaussian jitter on bytes (8% of std), 6% +/-1 flips on count features, 5% 'aggressive JIT' benign outliers pushed into suspicious ranges, 8% 'quiet malware' samples with signals shrunk 70% - classes deliberately overlap.
- Pipeline: RobustScaler (median/IQR - heavy-tailed byte counts) + HistGradientBoostingClassifier(max_iter=400, max_depth=6, lr=0.06, l2=0.5, class_weight='balanced', seed 42); RandomForest(600 trees, depth 14, balanced_subsample) available as alternate.
- Evaluation: 5-fold StratifiedKFold cross_val_predict (accuracy, balanced accuracy, precision, recall, F1, ROC-AUC, confusion matrix, classification report) + permutation importance (20 repeats) on a stratified 20% holdout; final production fit refits on 100% of the 6,000 samples and pickles to shadowmem_model.pkl.

- Class imbalance: handled via class_weight balancing at train time, matching the ~78/22 benign/malicious synthetic base rate.

4.4 Dashboard UI (embedded in app.py; standalone twin in index.html)

- Aesthetic: dark 'Tier-1 SOC' HUD - Tailwind CDN, Plus Jakarta Sans / JetBrains Mono, scanline overlay, laser sweep loading animation, radar scope canvas, DEFCON-style posture gauge with SVG ring.
- Function: summary stat cards (scanned, anomalies, JIT count, posture), filter tabs (ALL / MALICIOUS / SUSPICIOUS / JIT / CLEAN), search box (name or PID, '/' hotkey), sortable-by-score process table, per-process 'Forensic Dossier' modal showing the 7-vector breakdown, SOC recommendation, MITRE tag (T1055/T1620), and Isolate/Dump/Export buttons.
- Extras: RING-0 telemetry ticker with pause/clear, JSON report export button, toast notifications, error banner for scan failures (503 when memoryscanner missing, 500 when a scan raises).
- Honesty note: the modal's 'Extracted In-Memory Strings and Hex Preview' drawer shows a static hardcoded hex dump - it is cosmetic, not read from the process. Isolate/Dump buttons are not wired to any backend action.

4.5 Safe test harness (test_injector.py)

- Allocates 131,072 bytes with VirtualAlloc(MEM_COMMIT|MEM_RESERVE, PAGE_EXECUTE_READWRITE) in its own process - exactly the footprint the scanner flags.
- Writes b'\x90' * 131071 + b'\xC3' (NOP sled ending in RET) - never executed, so it is safe by construction; explicitly performs no OpenProcess/VirtualAllocEx/CreateRemoteThread.
- Spawns a 'ShadowMem-BenignTelemetryWorker' thread so thread metrics look live; prints an expected-telemetry banner (PID, base address, protection, type) for demo narration; VirtualFree(MEM_RELEASE) teardown on Ctrl+C.
- Demo flow it enables: run injector in terminal 1 -> run detecter.py or the dashboard scan -> observe the python.exe process flagged MALICIOUS with high score -> Ctrl+C and re-scan to show the threat cleared.

5. What Has Been Done

Everything below is verified by reading the code on disk (no git history exists, so this reflects shipped code rather than commit history):

Achieved	Detail
Working real scanner	User-mode VAD walk via VirtualQueryEx over all accessible processes; RWX/unbacked metrics; psutil integration; graceful AccessDenied handling
Trained ML artifact	shadowmem_model.pkl checked in; reproducible trainer with 7 documented forensic sub-classes, noise/overlap realism, CV metrics, permutation importance
Full detection API	FastAPI /api/scan returning scored, sorted process list + posture summary; model-missing fallback heuristic; structured error handling
SOC dashboard (real)	Embedded single-file UI: scan button, animated states, stat cards, filters, search, dossier modal, telemetry ticker, JSON export
CLI detector	detecter.py table output for quick terminal demos and validation
Safe live-fire demo	test_injector.py creates genuine RWX footprint safely; documented baseline-scan / inject / detect / teardown sequence
WebSocket streaming	Now in app.py: /ws/alerts broadcasts live alerts and memory_map frames from real scan data, with MITRE-tagged simulate scenarios (server.py retired to tests/mocks/ as the mock reference)
Backend unification (complete)	The former P0 'unify the two backends' backlog item is done: app.py hosts /ws/alerts and the memory_map visualizer fed by real scans, server.py retired to tests/mocks/ as a test-only mock - no port conflict
Launcher + docs	launch_shadowmem.bat one-click start; make_guide.py generates a full presentation/viva master guide (speaker scripts, Q&A;, demo sequence)

6. What Needs To Be Done (Prioritized Backlog)

Priority: P0 = blocks credibility or core correctness; P1 = high-value functionality; P2 = polish/hardening. Suggested session order follows the letters.

Pri	Item	Detail
P0	A. Implement features 5 and 7 for real	thread_unbacked_start and vad_protection_anomaly are hardcoded 0.0 in BOTH app.py and detecter.py, so the model only ever sees 5 of 7 features. Need NtQuerySystemInformation/NtQueryInformationThread (or psutil's threads with start-address via ctypes) to resolve thread start addresses against mapped regions, and either ETW/kernel telemetry or periodic re-protection diffing to detect RW->RX/RWX transitions.
P0	C. Fix doc-vs-code mismatches	make_guide.py's viva guide claims SMOTE and XGBoost; the pipeline actually uses class_weight balancing with HistGradientBoosting/RandomForest, and claims MZ/PE header carving and sub-2% CPU / 97.4% accuracy figures that are not implemented or measured anywhere. Either implement or reword before any defense/demo.
P1	D. Unbacked-region content carving	Read the first bytes of unbacked executable regions (ReadProcessMemory) and flag MZ/PE signatures in MEM_PRIVATE memory - the strongest reflective-DLL signal, currently claimed but absent. Also disambiguates JIT heaps (no PE headers) from injected PEs.
P1	E. Wire the dossier actions	modalIsolateBtn (suspend process / NtSuspendProcess) and modalDumpBtn (real strings/hexdump from flagged regions via ReadProcessMemory) currently do nothing; replace the static hex drawer with live extraction.
P1	F. Harden JIT allowlist	Name-only matching is spoofable. Verify full image path under known install dirs and/or Authenticode signature before applying the 25-point cap; add an explicit UI badge when verification fails.
P1	G. Project hygiene	requirements.txt (fastapi, uvicorn, scikit-learn, numpy, pandas, psutil, reportlab), README with run instructions, pytest suite (unit-test evaluate_process thresholds, JIT cap, fallback heuristic; mock memoryscanner for /api/scan), and a smoke test for the trainer producing a loadable pickle.
P2	H. Benchmark harness	Measure real per-scan latency, per-process overhead, and CPU footprint; produce numbers to back (or correct) the performance claims in the guide.
P2	I. Model realism pass	Validate the classifier against real or public telemetry (e.g. labeled dumps, MITRE ATT&CK; evaluations data); tune thresholds on a PR curve instead of the fixed 70/35; persist CV metrics alongside the pickle.
P2	J. Security hardening	server.py CORS is allow_origins=['*'] - restrict or drop before merging; sanitize all process names before rendering (currently trusted into the DOM); add rate limiting and an explicit admin-privilege check with a friendly error.
P2	K. Roadmap items from the guide	Linux eBPF port, Windows kernel driver (WDM) for zero-latency trapping, DL on carved memory streams - explicitly future scope, not session work.

7. Known Gaps and Honest Limitations

- Feature honesty: only 5 of the 7 advertised features are computed (see 6A); the UI presents the full 7-vector breakdown, so features 5 and 7 always display as zero.
- unbacked_exec_count includes PAGE_EXECUTE_READ pages (0x20), not just RWX - defensible forensically (RX private memory is still suspicious) but the dashboard label 'RWX pages' oversells it and the column semantics differ from train data intuition.
- User-mode only: cannot see kernel pools, and processes it cannot open (many system processes without admin) are silently skipped, so 'total scanned' under-reports.
- Snapshot semantics: a scan is a single instant - a beacon that unmaps right after the walk, or a transition happening mid-scan, can be missed; no continuous monitoring loop or delta detection exists yet.
- Synthetic-only model validation: CV numbers come from the same generator that made the training data; treat all accuracy claims as unproven until validated on external data.
- index.html vs embedded UI drift: two dashboards with similar purpose will diverge; resolving via 6B is planned.
- Windows-only and admin-recommended; the demo stack also assumes port 8000 is free.

8. How To Run

Task	Command	Notes
Launch real stack	.venv\Scripts\activate -> python -m uvicorn app:app --reload --host 127.0.0.1 --port 8000 (or just launch_shadowmem.bat)	Dashboard at http://127.0.0.1:8000; click 'Scan System Memory'
Retrain model	python train_model.py	Regenerates shadowmem_model.pkl (~6,000 samples, prints CV metrics)
CLI scan	python detecter.py	Terminal table; run as admin for maximum process coverage
Live-fire demo	Terminal 1: python test_injector.py (keep running). Terminal 2: python detecter.py (or dashboard scan). Expect the injector PID flagged MALICIOUS.	Ctrl+C the injector, re-scan, threat clears
Mock demo stack	cd tests/mocks && python -m uvicorn server:app --port 8082; open index.html; then python trigger_test_event.py	Test-only mock - do not run alongside app.py on the same port

Environment note: the .venv contains fastapi, uvicorn, scikit-learn, numpy, pandas, psutil, reportlab (used by this script) on Python 3.14. There is no requirements.txt yet - see backlog item G.

9. Appendix - Compact Fact Sheet for Session Planning

Fact	Value
Feature vector order (must match pickle)	0 rwx_count, 1 unbacked_exec_count, 2 total_rwx_bytes, 3 num_threads, 4 thread_unbacked_start, 5 is_known_jit_parent, 6 vad_protection_anomaly
Verdict thresholds	Malicious >= 70.0 | Suspicious >= 35.0 | Clean < 35.0 | JIT engines capped at 25.0
Model contract	shadowmem_model.pkl = sklearn Pipeline(RobustScaler, HistGradientBoostingClassifier); call predict_proba([[7 features]]); malicious prob = column[-1] * 100
app.py endpoints	GET / (embedded dashboard HTML) | GET /api/scan -> {summary{total_scanned, anomalies_detected, malicious_count, suspicious_count, jit_engine_count, clean_count, system_posture, model_loaded}, processes[{pid, name, rwx_count, unbacked_exec_count, total_rwx_bytes, allocated_kb, num_threads, threat_score, status, jit_engine, model_used}]}
server.py (mock fixture)	Not a live server: preserved only under tests/mocks/ for testing; WebSocket streaming (live alerts and memory_map broadcasts) is hosted entirely in app.py
Process record (scanner)	{pid, name, rwx_count, unbacked_exec_count, total_rwx_bytes, num_threads, error}
JIT allowlist	chrome, msedge, firefox, code, discord, antigravity, slack, teams, electron, brave, opera, vivaldi (.exe)
Ports	app.py = 8000 (primary/production) | tests/mocks/server.py = 8082 (test mock only, no conflict)
Key UI element IDs (app.py)	scanBtn, exportReportBtn, resultsBody, forensicModal, modalIsolateBtn, modalDumpBtn, stringsDrawer, kernelTicker, radarCanvas, postureGaugeCircle
Win32 constants in play	PAGE_EXECUTE_READWRITE 0x40, PAGE_EXECUTE_READ 0x20, MEM_COMMIT 0x1000, MEM_PRIVATE 0x20000, PROCESS_QUERY_INFORMATION 0x0400, PROCESS_VM_READ 0x0010
Trainer entry points	train_model.build_synthetic_dataset(n_total=6000), build_pipeline(model_type='hgb'|'rf'), evaluate_cv, report_feature_importance, main()
Demo scenarios (server.py)	cobalt_strike_reflective_injection -> T1620, process_hollowing -> T1055.012, dll_injection -> T1055.001
Fastest credibility win	Backlog 6A (real features 5 and 7) + 6D (MZ/PE carving) together make the demo claims true; 6B unifies the product story

Suggested session plan for an AI assistant: (1) read this PDF; (2) start with P0 items A-C in order - A gives the model its missing eyes, B merges the product, C aligns claims with reality; (3) then P1 items D-G; (4) run test_injector.py end-to-end after each milestone to prove detection still works. Treat the synthetic-data caveat in Section 7 as a standing constraint on any accuracy claims.